23CSE211

Design and Analysis of Algorithms

Lab-2 04/12/2025

Submitted to

Dr SS Chakravarthi

Submitted by

Ruhan T

CH.SC.U4CSE24240

Lab-2

Sorting Techniques

1. Bubble sort

```
#include <stdio.h>
void bubbleSort(int arr[], int n) {
int i, j, temp;
for (i = 0; i < n - 1; i++) {
for (j = 0; j < n - i - 1; j++) {
if (arr[j] > arr[j + 1]) {
temp = arr[j];
arr[j] = arr[j + 1];
arr[j + 1] = temp;
}
}
}
}
int main() {
int arr[] = {64, 34, 25, 12, 22, 11, 90};
int n = 7;
int i;
printf("Original array: \n");
for (i = 0; i < n; i++) {
printf("%d ", arr[i]);
}
printf("\n");
bubbleSort(arr, n);
printf("Sorted array: \n");
for (i = 0; i < n; i++) {
printf("%d ", arr[i]);
}
return 0;
}
```

2. Merge Sort

```
#include <stdio.h>
void merge(int arr[], int l, int m, int r) {
int i, j, k;
int n1 = m - l + 1;
int n2 = r - m;
```

```
int L[n1], R[n2];
for (i = 0; i < n1; i++) L[i] = arr[l + i];
for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
i = 0; j = 0; k = l;
while (i < n1 && j < n2) {
if (L[i] <= R[j]) {
arr[k] = L[i];
i++;
} else {
arr[k] = R[j];
j++;
}
k++;
}
while (i < n1) {
arr[k] = L[i];
i++; k++;
}
while (j < n2) {
arr[k] = R[j];
j++; k++;
}
}
void mergeSort(int arr[], int l, int r) {
if (l < r) {
int m = l + (r - l) / 2;
mergeSort(arr, l, m);
mergeSort(arr, m + 1, r);
merge(arr, l, m, r);
}
}
int main() {
int arr[] = {12, 11, 13, 5, 6, 7};
int n = 6;
int i;
mergeSort(arr, 0, n - 1);
printf("Sorted array: \n");
for (i = 0; i < n; i++) {
printf("%d ", arr[i]);
}
return 0;
}
```

```
ruhaaa@Master:~/Documents/Linux$ gcc mergesort.c -o mergesort
ruhaaa@Master:~/Documents/Linux$ ./mergesort
Sorted array:
5 6 7 11 12 13 ruhaaa@Master:~/Documents/Linux$
```

3. Selection Sort

```
#include <stdio.h>
void selectionSort(int arr[], int n) {
int i, j, min_idx, temp;
for (i = 0; i < n - 1; i++) {
min_idx = i;
for (j = i + 1; j < n; j++) {
if (arr[j] < arr[min_idx])
min_idx = j;
}
temp = arr[min_idx];
arr[min_idx] = arr[i];
arr[i] = temp;
}
}
int main() {
int arr[] = {64, 25, 12, 22, 11};
int n = 5;
int i;
printf("Sorted array: \n");
selectionSort(arr, n);
for (i = 0; i < n; i++) {
printf("%d ", arr[i]);
}
return 0;
}
```

```
ruhaaa@Master:~/Documents/Linux$ gcc selectionsort.c -o selectionsort
ruhaaa@Master:~/Documents/Linux$ ./selectionsort
Sorted array:
11 12 22 25 64 ruhaaa@Master:~/Documents/Linux$
```

4. Insertion Sort

```
#include <stdio.h>
void insertionSort(int arr[], int n) {
int i, key, j;
for (i = 1; i < n; i++) {
key = arr[i];
j = i - 1;
while (j >= 0 && arr[j] > key) {
arr[j + 1] = arr[j];
j = j - 1;
}
arr[j + 1] = key;
}
}
int main() {
int arr[] = {12, 11, 13, 5, 6};
int n = 5;
```

```
int i;
insertionSort(arr, n);
printf("Sorted array: \n");
for (i = 0; i < n; i++) {
printf("%d ", arr[i]);
}
return 0;
}
```

```
ruhaaa@Master:~/Documents/Linux$ gcc insertionsort.c -o insertionsort
ruhaaa@Master:~/Documents/Linux$ ./insertionsort
Sorted array:
5 6 11 12 13 ruhaaa@Master:~/Documents/Linux$ gcc insertionsort.c -o insertionso
```

5. Bucket Sort

```
#include <stdio.h>
void bucketSort(float arr[], int n) {
float b[10][100];
int bucketCounts[10]; // Keeps track of how many elements are in each bucket
int i, j, k;
for(i = 0; i < 10; i++) {
bucketCounts[i] = 0;
}
for (i = 0; i < n; i++) {
int bi = (int)(n * arr[i]);
if (bi > 9) bi = 9;
b[bi][bucketCounts[bi]] = arr[i];
bucketCounts[bi]++;
}
for (i = 0; i < 10; i++) {
if (bucketCounts[i] > 1) {
for (j = 1; j < bucketCounts[i]; j++) {
float key = b[i][j];
k = j - 1;
while (k >= 0 && b[i][k] > key) {
b[i][k + 1] = b[i][k];
k--;
}
b[i][k + 1] = key;
}
}
}
int index = 0;
for (i = 0; i < 10; i++) {
for (j = 0; j < bucketCounts[i]; j++) {
arr[index++] = b[i][j];
}
}
}
```

```
int main() {
float arr[] = {0.89, 0.56, 0.65, 0.12, 0.66, 0.34};
int n = 6;
int i;
printf("Original array: \n");
for (i = 0; i < n; i++) {
printf("%.2f ", arr[i]);
}
printf("\n");
bucketSort(arr, n);
printf("Sorted array: \n");
for (i = 0; i < n; i++) {
printf("%.2f ", arr[i]);
}
return 0;
}
```

```
ruhaaa@Master:~/Documents/Linux$ gcc bucketsort.c -o bucketsort
ruhaaa@Master:~/Documents/Linux$ ./bucketsort
Original array:
0.89 0.56 0.65 0.12 0.66 0.34
Sorted array:
0.12 0.34 0.56 0.65 0.66 0.89 ruhaaa@Master:~/Documents/Linux$
```